Remapping Keys for ColemakVIM on MacOS

Rohit Goswami · 2021-07-23 · workflow · macos · tools

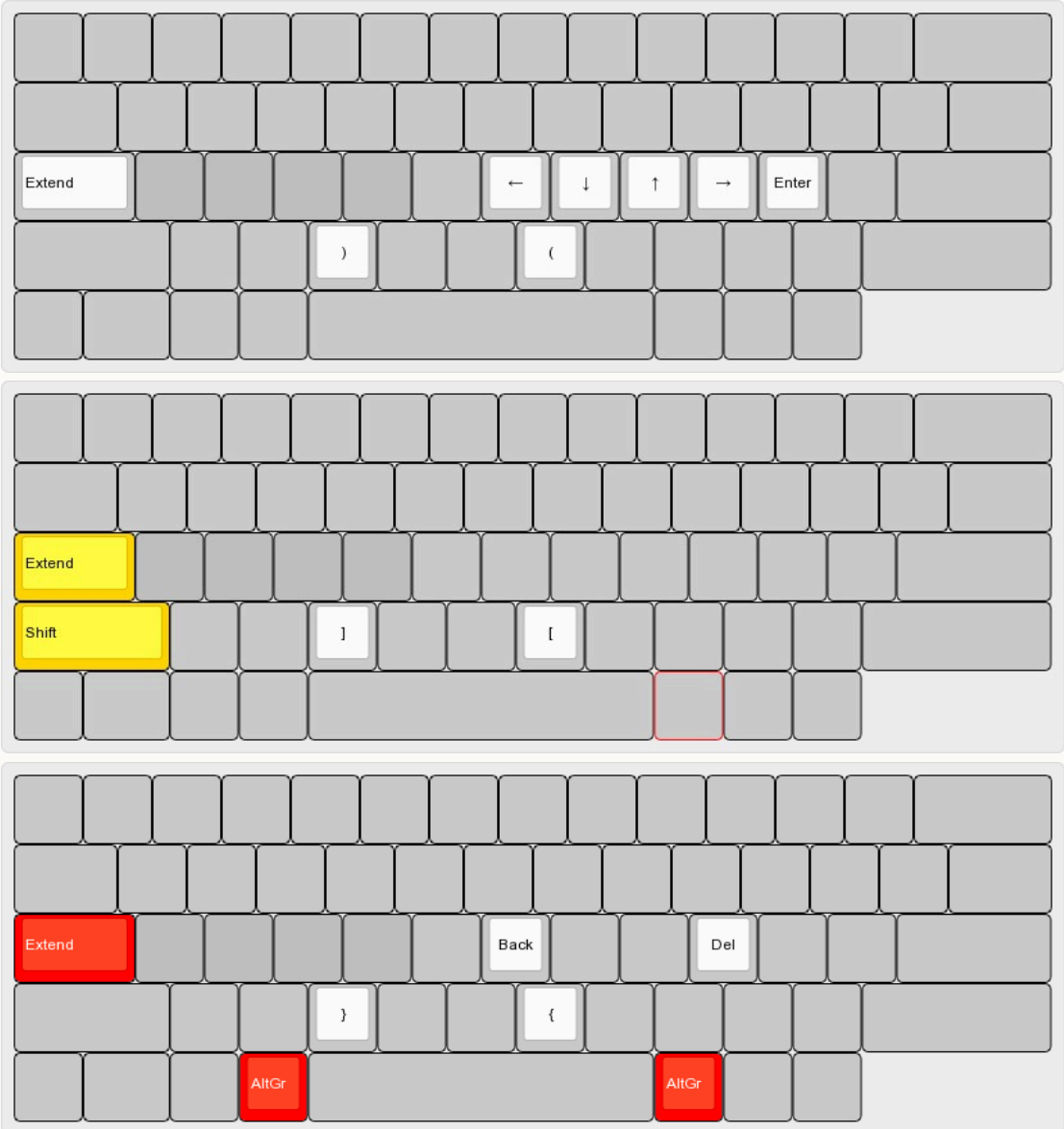
> *Struggling to emulate `klfc` for VIM Colemak bindings on Darwin (macOS) systems with Hammerspoon and Karabiner*

Background

I have mentioned in the past my customized Colemak dotfiles which I used with a customized keyboard layout. Unfortunately, the `.keylayout` system of MacOS is far more primitive than the elegant `klfc` setup [^fn:1]. For an understanding of what we are trying to get at, the following poorly made video will suffice.

VIM Colemak Extensions

Just as a reminder, my setup (or `hzColemak`) consists of an augmented VIM workflow, as shown below, and described in my previous post.



Tools

There are essentially a few options:

Manually write `.keylayout`

These then go in `$HOME/Library/Keyboard\ Layouts`

Use Ukelele

The incredibly poorly named (for search purposes) versatile tool is able to ease the pain slightly for writing `.keylayout` files

Use Karabiner Elements

This seems to be closer to AutoHotKey and the like, runs in the background and actively intercepts keys based on `json` configurations; though there seems to be a more rational method (a `hidutil` remapping file generator) for newer kernels [^fn:2]

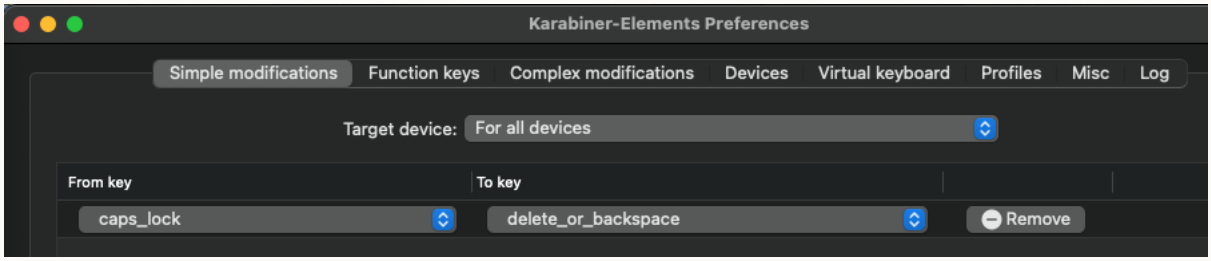
Script things with Hammerspoon

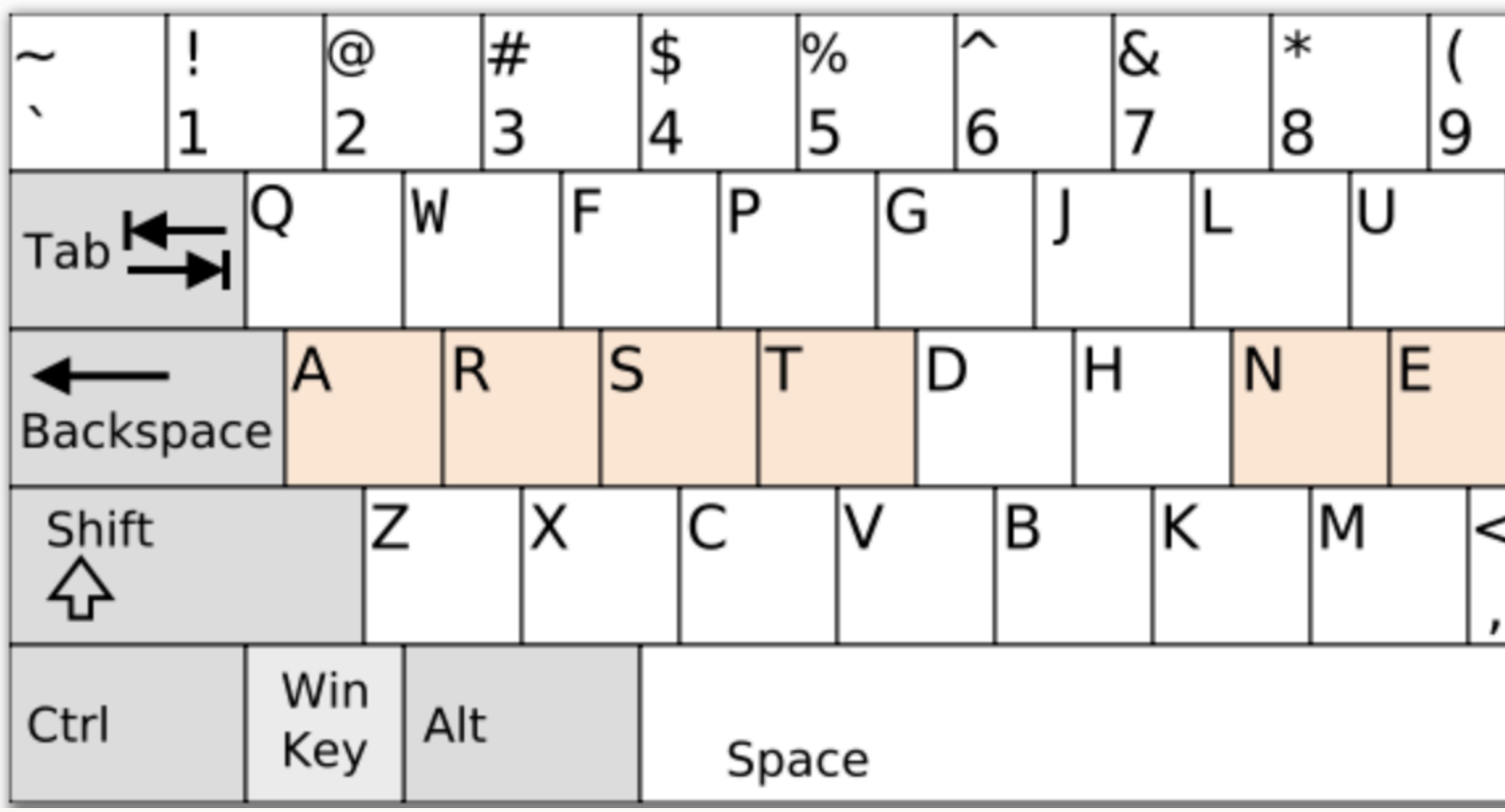
Uses a `lua` engine to interact with the system, can be configured for the most part with Fennel using Spacehammer

Now of the four, I had a predilection to move towards manually writing, with the help of Ukelele. However, evidently, there is no real way to remove the stickiness from the Caps Lock key. It can either be remapped using **system settings** [^fn:3] to one of the other modifier keys, but **not to Extend**. The closest possible solution would be to do a very awkward `Esc` based layout. Also, rapid prototyping was out of the question, since Ukelele requires a log-out log-in cycle to set things up. Of Ukelele and manually writing things then, nothing more need be said.

Karabiner

This left me considering `json` re-writer. I have been using the basic Colemak layout with a simplistic Karabiner `caps` to `delete` for a while now, which allows for a standards compliant Colemak experience, but extending this like I needed was a little bit of a struggle.





Apparently it is possible to overload the keyboard system with a “Hyper” key [^fn:4], which is the closest to Extend.

This is setup through a `karabiner.json` file, since it appears that the “Complex modifications” referred to in the GUI (Fig. 2) don’t really allow for more than downloading rules off of the internet [^fn:5], like the one below.

```
"complex_modifications": {
                "rules": [
                    {
                        "manipulators": [
                            {
                                "description": "Change caps_lock to command+control+option+shift. Escape if no other key used.",
                                "from": {
                                    "key_code": "caps_lock",
                                    "modifiers": {
                                        "optional": [
                                            "any"
                                        ]
                                    }
                                },
                                "to": [
                                    {
                                        "key_code": "left_shift",
                                        "modifiers": [
                                            "left_command",
                                            "left_control",
                                            "left_option"
                                        ]
                                    }
                                ],
                                "to_if_alone": [
                                    {
                                        "key_code": "escape",
                                        "modifiers": {
                                            "optional": [
                                                "any"
                                            ]
                                        }
                                    }
                                ],
                                "type": "basic"
                            }
                        ]
                    }
                ]
            },
```

Extending this is not pleasant, so we won't go forward with this approach at all.

We will still need a bit of this unfortunately, so this will stick around. In particular we need to have a simple mapping, from `caps_lock` to one of the un-mapped function keys, `f18` or `f19` or some such.

Effectively, this can be done with the GUI, but we prefer the configuration snippet makes it far more reproducible:

```
{
    "type": "basic",
    "from": {
        "key_code": "caps_lock"
    },
    "to":
    {
        "key_code": "f19"
    }
}
```

Note that we **do not** need to define the `hyper` key to be a series of keystrokes or anything like that here with the `complex_modifications` in this configuration. The point of using `complex_modifications` is to not overwrite the Karabiner defaults. The following modifications also allow us to keep CAPS functionality bound to simultaneously pressing left and right shift together[^fn:6].

```
{
  "title": "CAPS to fake hyper (f19) and Shift CAPS",
  "rules": [
    {
      "description": "CAPS_LOCK : (HYPER)",
      "manipulators": [
        {
          "from": {
            "key_code": "caps_lock",
            "modifiers": {
              "optional": ["any"]
            }
          },
          "to":
            {
              "key_code": "f19"
            },
          "type": "basic"
        }
      ]
    },
    {
      "description": "Toggle CAPS_LOCK with LEFT_SHIFT + RIGHT_SHIFT",
      "manipulators": [
        {
          "from": {
            "key_code": "left_shift",
            "modifiers": {
              "mandatory": ["right_shift"],
              "optional": ["caps_lock"]
            }
          },
          "to": [
            {
              "key_code": "caps_lock"
            }
          ],
          "to_if_alone": [
            {
              "key_code": "left_shift"
            }
          ],
          "type": "basic"
        },
        {
          "from": {
            "key_code": "right_shift",
            "modifiers": {
              "mandatory": ["left_shift"],
              "optional": ["caps_lock"]
            }
          },
          "to": [
            {
              "key_code": "caps_lock"
            }
          ],
          "to_if_alone": [
            {
              "key_code": "right_shift"
            }
          ],
          "type": "basic"
        }
      ]
    }
  ]
}
```

Hammerspoon

Not that an operating system should need a scripting engine, but apparently Hammerspoon is worth looking into fox better control over the entire OS [^fn:7]. This made it more attractive than writing a hidden configuration file for Karabiner.

The first thing we will need in our `init.lua` after installing the Hammerspoon application is one to reload the configuration automatically:

```
-- Reload config when any lua file in config directory changes
-- From https://gist.github.com/prenagha/1c28f71cb4d52b3133a4bff1b3849c3e
function reloadConfig(files)
    doReload = false
    for _,file in pairs(files) do
        if file:sub(-4) == '.lua' then
            doReload = true
        end
    end
    if doReload then
        hs.reload()
    end
end
local myWatcher = hs.pathwatcher.new(os.getenv('HOME') .. '/.hammerspoon/', reloadConfig):start()
hs.alert.show('Config loaded')
```

An alternative to using `Karabiner` to map `f19` is to use the foundation remapping plugin which needs to be placed in `$HOME/.hammerspoon` with the following added to our `init.lua`:

```
-- cd ~/.hammerspoon/ && wget https://raw.githubusercontent.com/hetima/hammerspoon-foundation_remapping/master/foundation_remapping.lua
-- init.lua
local FRemap = require('foundation_remapping')
local remapper = FRemap.new()

remapper:remap('capslock', 'f19')
remapper:register()
```

The only problem with this is that it must be `unregistered` carefully. The Karabiner method is easier overall.

Anyway, once `f19` has been bound, one way or another, we need to setup the hyper key itself[^fn:8]:

```
-- init.lua
-- A global variable for the Hyper Mode
hyper = hs.hotkey.modal.new({}, 'F18')

-- Enter Hyper Mode when F19 (Hyper/Capslock) is pressed
function enterHyperMode()
  hyper.triggered = false
  hyper:enter()
  hs.alert.show('Hyper on')
end

-- Leave Hyper Mode when F19 (Hyper/Capslock) is pressed,
function exitHyperMode()
  hyper:exit()
  hs.alert.show('Hyper off')
end

-- Bind the Hyper key
f19 = hs.hotkey.bind({}, 'F19', enterHyperMode, exitHyperMode)
f19cmd = hs.hotkey.bind({'cmd'}, 'F19', enterHyperMode, exitHyperMode)
```

The alerts can help while setting up muscle memory, but they are optional. We also map `cmd+f19` to prevent the `cmd+h` hide application annoyance.

Remapping Keys

The basic idea is to set a function which executes only when `hyper:enter()` is true. One thing to remember is that the signature for `hs.eventtap.keyStroke` has an optional `delay` which defaults to 200ms and is ridiculous. This means we need to explicitly set `0` for the delay.

Basic Movements

These correspond to the to the top level extend layer. We will disect one example here and refer to a later section for the details.

```
-- h - move left {{{3
function left() hs.eventtap.keyStroke({}, "Left", 0) end
hyper:bind({}, 'h', left, nil, left)
-- }}}3
```

Essentially, when `hyper` is active, then tapping `h` activates the `keyStroke`. The remaining `hnei` movements are similar. For the carriage return case we can go for a slightly more interesting option.

```
-- o - open new line below cursor {{{3
hyper:bind({}, 'o', nil, function()
    local app = hs.application.frontmostApplication()
    if app:name() == "Finder" then
        hs.eventtap.keyStroke({"cmd"}, "o", 0)
    else
        hs.eventtap.keyStroke({}, "Return", 0)
    end
end)
-- }}}3
```

We can effectively setup application specific `keyStrokes` which will help in the long run.

Extend Layers

These are implemented in much the same way, since modifiers can be called in the `bind` function.

```
-- cmd+h - delete character before the cursor {{{3
local function delete()
    hs.eventtap.keyStroke({}, "delete", 0)
end
hyper:bind({"cmd"}, 'h', delete, nil, delete)
-- }}}3
```

VIM Extras

Since we have a whole scripting engine anyway, we might as well use it to get some additional mileage not possible from our older keyboard layout.

```
-- w - move to next word {{{3
function word() hs.eventtap.keyStroke({"alt"}, "Right", 0) end
hyper:bind({}, 'w', word, nil, word)
-- }}}3
```

These can be extended further into whole pseudo-VIM approach.

All together

For this post, we opted for a minimal Hammerspoon setup which ended up with a monolithic setup defined in `init.lua` files as follows:

```
-- Reload config when any lua file in config directory changes
-- From https://gist.github.com/prenagha/1c28f71cb4d52b3133a4bff1b3849c3e
function reloadConfig(files)
    doReload = false
    for _,file in pairs(files) do
        if file:sub(-4) == '.lua' then
            doReload = true
        end
    end
    if doReload then
        hs.reload()
    end
end
local myWatcher = hs.pathwatcher.new(os.getenv('HOME') .. '/.hammerspoon/', reloadConfig):start()
hs.alert.show('Config loaded')

-- A global variable for the Hyper Mode
hyper = hs.hotkey.modal.new({}, 'F18')

-- Enter Hyper Mode when F19 (Hyper/Capslock) is pressed
function enterHyperMode()
  hyper.triggered = false
  hyper:enter()
  hs.alert.show('Hyper on')
end

-- Leave Hyper Mode when F19 (Hyper/Capslock) is pressed,
-- send ESCAPE if no other keys are pressed.
function exitHyperMode()
  hyper:exit()
  -- if not hyper.triggered then
  --   hs.eventtap.keyStroke({}, 'ESCAPE')
  -- end
  hs.alert.show('Hyper off')
end

-- Bind the Hyper key
f19 = hs.hotkey.bind({}, 'F19', enterHyperMode, exitHyperMode)

-- Vim Colemak bindings (hzColemak)
-- Basic Movements {{{2

-- h - move left {{{3
function left() hs.eventtap.keyStroke({}, "Left", 0) end
hyper:bind({}, 'h', left, nil, left)
-- }}}3

-- n - move down {{{3
function down() hs.eventtap.keyStroke({}, "Down", 0) end
hyper:bind({}, 'n', down, nil, down)
-- }}}3

-- e - move up {{{3
function up() hs.eventtap.keyStroke({}, "Up", 0) end
hyper:bind({}, 'e', up, nil, up)
-- }}}3

-- i - move right {{{3
function right() hs.eventtap.keyStroke({}, "Right", 0) end
hyper:bind({}, 'i', right, nil, right)
-- }}}3

-- ) - right programming brace {{{3
function rbroundL() hs.eventtap.keyStrokes("(") end
hyper:bind({}, 'k', rbroundL, nil, rbroundL)
-- }}}3

-- ) - left programming brace {{{3
function rbroundR() hs.eventtap.keyStrokes(")") end
hyper:bind({}, 'v', rbroundR, nil, rbroundR)
-- }}}3

-- o - open new line below cursor {{{3
hyper:bind({}, 'o', nil, function()
    local app = hs.application.frontmostApplication()
    if app:name() == "Finder" then
        hs.eventtap.keyStroke({"cmd"}, "o", 0)
    else
        hs.eventtap.keyStroke({}, "Return", 0)
    end
end)
-- }}}3

-- Extend+AltGr layer
-- Delete {{{3

-- cmd+h - delete character before the cursor {{{3
local function delete()
    hs.eventtap.keyStroke({}, "delete", 0)
end
hyper:bind({"cmd"}, 'h', delete, nil, delete)
-- }}}3

-- cmd+i - delete character after the cursor {{{3
local function fndelete()
    hs.eventtap.keyStroke({}, "Right", 0)
    hs.eventtap.keyStroke({}, "delete", 0)
end
```

```
hyper:bind({"cmd"}, 'i', fndelete, nil, fndelete)
-- }}}3

-- ) - right programming brace {{{3
function rbcurlyL() hs.eventtap.keyStrokes("{") end
hyper:bind({"cmd"}, 'k', rbcurlyL, nil, rbcurlyL)
-- }}}3

-- ) - left programming brace {{{3
function rbcurlyR() hs.eventtap.keyStrokes("}") end
hyper:bind({"cmd"}, 'v', rbcurlyR, nil, rbcurlyR)
-- }}}3

-- Extend+Shift

-- ) - right programming brace {{{3
function rbsqrL() hs.eventtap.keyStrokes("[") end
hyper:bind({"shift"}, 'k', rbsqrL, nil, rbsqrL)
-- }}}3

-- ) - left programming brace {{{3
function rbsqrR() hs.eventtap.keyStrokes("]") end
hyper:bind({"shift"}, 'v', rbsqrR, nil, rbsqrR)
-- }}}3

-- Special Movements
-- w - move to next word {{{3
function word() hs.eventtap.keyStroke({"alt"}, "Right", 0) end
hyper:bind({}, 'w', word, nil, word)
-- }}}3

-- b - move to previous word {{{3
function back() hs.eventtap.keyStroke({"alt"}, "Left", 0) end
hyper:bind({}, 'b', back, nil, back)
-- }}}3
```

Conclusions

It has been very restrictive to not be able to use the keyboard layout I spent years crafting. In that regard, this post is a success story, even with the awkwardness of the implementation. On the other hand, it is baffling to see the lack of good FOSS tools on this ecosystem [^fn:9], but that is to be expected perhaps. For my purposes right now, this monolithic `init.lua` is enough, and the entire configuration corresponds to this commit in my Dotfiles. However, Hammerspoon is one of the more promising configuration systems especially considering the more complete VIM bindings which exist in the community and would bear a second look. It would make sense to look into using Spacehammer and more VIM bindings sometime soon.